

Toy HTTP Server

A developer-focused, phase-by-phase specification for building an HTTP/1.1 server from scratch with Rust's standard library.

RUNTIME Rust stdlib only	PROTOCOL HTTP/1.1 over TCP	LEARNING PATH 7 build phases
--	--	--

Contents

Project Overview	->
Phase 1: TCP Listener	->
Phase 2: HTTP Request Parsing	->
Phase 3: HTTP Response	->
Phase 4: Routing	->
Phase 5: Multi-threading	->
Phase 6: Thread Pool	->
Phase 7: Polish and Hardening	->
Final Checklist	->
Suggested Extensions (Post-MVP)	->

SECTION 01

Project Overview

Build a fully functional HTTP/1.1 server from scratch in Rust using only the standard library. No frameworks, no crates.io dependencies for core functionality. The server will accept real TCP connections, parse HTTP requests, route them to handler functions, and return proper HTTP responses — all while handling multiple clients concurrently via a custom thread pool.

Goals

- Deepen understanding of Rust beyond smart contract development
- Learn TCP socket programming and network I/O
- Master ownership, borrowing, and lifetimes in a real-world context
- Understand concurrency primitives: threads, channels, mutexes, Arc
- Build intuition for how web frameworks (Actix, Axum, Warp) work under the hood

Target File Structure

```
toy_http_server/
├── Cargo.toml
├── GUIDE.md
├── src/
│   ├── main.rs           # Entry point – starts the server
│   ├── server.rs         # TcpListener, connection acceptance loop
│   ├── request.rs        # HTTP request parsing
│   ├── response.rs       # HTTP response construction and serialization
│   ├── router.rs         # URL-to-handler routing
│   ├── handlers.rs       # Endpoint handler functions
│   └── thread_pool.rs    # Fixed-size thread pool with job queue
└── tests/
    ├── request_test.rs   # Unit tests for request parsing
    ├── response_test.rs  # Unit tests for response building
    ├── router_test.rs    # Unit tests for routing logic
    └── integration_test.rs # End-to-end tests with real TCP connections
```

Testing Strategy

Throughout the build, use `curl` as the primary client for manual testing:

```
curl -v http://localhost:8080/
curl -v -X POST -d "hello=world" http://localhost:8080/echo
curl -v http://localhost:8080/users/42
```

The `-v` flag shows the full HTTP exchange (request + response), which is invaluable for debugging.

SECTION 02

Phase 1: TCP Listener

Timeline: 1-2 days Difficulty: 3/10

Objective

Bind to a port, accept incoming TCP connections, read raw bytes from the stream, and echo them back. No HTTP understanding required yet — just raw network I/O.

Key Concepts

- `std::net::TcpListener` — binds to an address and listens for connections
- `std::net::TcpStream` — represents a single TCP connection (implements `Read` and `Write` traits)
- `std::io::Read` and `std::io::Write` traits — the standard interface for I/O in Rust
- `Result<T, E>` — every I/O operation can fail, so everything returns `Result`

Files to Create/Modify

`src/server.rs`

```
use std::net::TcpListener;

pub struct Server {
    listener: TcpListener,
}

impl Server {
    pub fn new(address: &str) -> Result<Self, std::io::Error> {
        let listener = TcpListener::bind(address)?;
        Ok(Self { listener })
    }

    pub fn run(&self) -> Result<(), std::io::Error> {
        // Accept connections in a loop
        // For each connection, read bytes and print them
    }
}
```

`src/main.rs`

```
mod server;

fn main() {
    let server = server::Server::new("127.0.0.1:8080").unwrap();
    server.run().unwrap();
}
```

Implementation Details

1. `TcpListener::bind("127.0.0.1:8080")` — binds to localhost on port 8080. Returns `Result<TcpListener, io::Error>`.

2. `listener.incoming()` — returns an iterator over `Result<TcpStream, io::Error>`. Each item is a new connection.
3. For each `TcpStream`, read into a `Vec<u8>` or `[u8; 1024]` buffer using `stream.read(&mut buffer)`.
4. Print the raw bytes as a UTF-8 string using `String::from_utf8_lossy(&buffer[..bytes_read])`.
5. Write a simple response back: `stream.write_all(b"HTTP/1.1 200 OK\r\n\r\nHello!")`.

What You'll See

Open a browser to `http://localhost:8080` or run `curl -v http://localhost:8080`. You'll see the raw HTTP request your client sends:

```
GET / HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.0
Accept: */*
```

This is the raw text protocol you'll be parsing in Phase 2.

Pitfalls

- **Blocking reads:** `stream.read()` blocks until data arrives. This is fine for now but will be a problem when handling multiple connections (Phase 5).
- **Port already in use:** If you kill the server and restart quickly, you may get "Address already in use." Either wait a few seconds or use a different port.
- **Partial reads:** A single `read()` call may not return the entire request. For now, a 1024-byte buffer is sufficient, but keep this in mind for Phase 2.

Acceptance Criteria

- ☐ Server binds to `127.0.0.1:8080` without errors
- ☐ Server accepts connections and prints raw request bytes to stdout
- ☐ Server sends back a hardcoded response that `curl` can display
- ☐ Server handles the client disconnecting gracefully (no panic)

SECTION 03

Phase 2: HTTP Request Parsing

Timeline: 2-3 days Difficulty: 5/10

Objective

Parse the raw bytes from a TCP stream into a structured `Request` type that your server can reason about. This is the most parsing-heavy phase.

Key Concepts

- HTTP/1.1 request format (RFC 7230)
- String slicing and `&str` vs `&[u8]` — HTTP is text-based, but TCP gives you bytes
- `str::split`, `str::trim`, `str::lines` — text processing methods
- `FromStr` trait — idiomatic parsing in Rust
- `Option` and `Result` for handling malformed input

HTTP/1.1 Request Format

```
GET /users/42?name=alice HTTP/1.1\r\n
Host: localhost:8080\r\n
Content-Type: application/x-www-form-urlencoded\r\n
Content-Length: 11\r\n
\r\n
hello=world
```

Broken down:

1. **Request line:** `METHOD SP PATH SP VERSION CRLF`
2. **Headers:** `Key: Value CRLF` (zero or more)
3. **Blank line:** `CRLF` (marks end of headers)
4. **Body:** Optional, length determined by `Content-Length` header

Where `SP` = space (`0x20`), `CRLF` = `\r\n` (`0x0D 0x0A`).

Files to Create

src/request.rs

```
#[derive(Debug, PartialEq)]
pub enum Method {
    GET,
    POST,
    PUT,
    DELETE,
    PATCH,
    HEAD,
    OPTIONS,
}

#[derive(Debug)]
pub struct Request {
    pub method: Method,
    pub path: String,
    pub query: Option<String>,
    pub version: String,
    pub headers: Vec<(String, String)>,
    pub body: Option<String>,
}

#[derive(Debug)]
pub enum ParseError {
    InvalidRequestLine,
    InvalidMethod,
    InvalidHeader,
    IncompleteRequest,
    InvalidUtf8,
}

impl Request {
    pub fn parse(raw: &[u8]) -> Result<Self, ParseError> {
        // 1. Convert bytes to &str
        // 2. Split on "\r\n\r\n" to separate headers from body
        // 3. Parse request line (first line)
        // 4. Parse headers (remaining lines before blank line)
        // 5. Extract body if present
    }
}
```

Implementation Details

Step 1 — Convert bytes to string:

```
let request_str = std::str::from_utf8(raw).map_err(|_| ParseError::InvalidUtf8)?;
```

Step 2 — Split headers from body:

```
let mut parts = request_str.splitn(2, "\r\n\r\n");
let header_section = parts.next().ok_or(ParseError::IncompleteRequest)?;
let body_section = parts.next();
```

Step 3 — Parse request line:

```
let mut lines = header_section.lines();
let request_line = lines.next().ok_or(ParseError::InvalidRequestLine)?;
let mut tokens = request_line.splitn(3, ' ');

let method_str = tokens.next().ok_or(ParseError::InvalidRequestLine)?;
let full_path = tokens.next().ok_or(ParseError::InvalidRequestLine)?;
let version = tokens.next().ok_or(ParseError::InvalidRequestLine)?;
```

Step 4 — Parse path and query string:

```
let (path, query) = match full_path.split_once('?') {
    Some((p, q)) => (p.to_string(), Some(q.to_string())),
    None => (full_path.to_string(), None),
};
```

Step 5 — Parse headers:

```
let mut headers = Vec::new();
for line in lines {
    if line.is_empty() { break; }
    let (key, value) = line.split_once(": ")
        .ok_or(ParseError::InvalidHeader)?;
    headers.push((key.to_string(), value.trim().to_string()));
}
```

Step 6 — Parse method:

```
let method = match method_str {
    "GET" => Method::GET,
    "POST" => Method::POST,
    "PUT" => Method::PUT,
    "DELETE" => Method::DELETE,
    "PATCH" => Method::PATCH,
    "HEAD" => Method::HEAD,
    "OPTIONS" => Method::OPTIONS,
    _ => return Err(ParseError::InvalidMethod),
};
```

Step 7 — Extract body:

```
let body = body_section.map(|b| b.to_string()).filter(|b| !b.is_empty());
```

Helper Method

Add a convenience method to look up headers:

```
impl Request {
    pub fn get_header(&self, name: &str) -> Option<&str> {
        self.headers.iter()
            .find(|(k, _)| k.eq_ignore_ascii_case(name))
            .map(|(_, v)| v.as_str())
    }
}
```

Pitfalls

- **Header names are case-insensitive.** `Content-Length` and `content-length` are the same. Use `eq_ignore_ascii_case` for comparisons.
- **Header values may have leading/trailing whitespace.** Always `trim()` header values.
- **Not all requests have a body.** GET requests typically don't. Check for `Content-Length` before trying to read a body.
- **The `\r\n\r\n` delimiter is critical.** If you split on `\n\n` instead, you'll break on Windows-style line endings.
- **Incomplete reads.** A client may send headers in one TCP segment and the body in another. For this toy server, a single `read()` with a large buffer (4096 bytes) will work for most cases. A production server would need to loop until it reads `Content-Length` bytes.

Acceptance Criteria

- `Request::parse` correctly parses a well-formed GET request
- `Request::parse` correctly parses a POST request with a body
- `Request::parse` returns appropriate errors for malformed input
- Query strings are correctly separated from the path
- Headers are accessible by name (case-insensitive lookup)
- Unit tests cover: valid GET, valid POST, missing headers, invalid method, incomplete request line

SECTION 04

Phase 3: HTTP Response

Timeline: 1-2 days Difficulty: 3/10

Objective

Build a `Response` type that can be constructed, configured, and serialized into a valid HTTP/1.1 response byte stream.

Key Concepts

- HTTP/1.1 response format
- Builder pattern — ergonomic API for constructing responses
- `Write` trait — serializing to a `TcpStream`
- Status codes and their meanings

HTTP/1.1 Response Format

```
HTTP/1.1 200 OK\r\n
Content-Type: text/html\r\n
Content-Length: 13\r\n
\r\n
Hello, World!
```

Broken down:

1. **Status line:** `VERSION SP STATUS_CODE SP REASON_PHRASE CRLF`
2. **Headers:** `Key: Value CRLF` (zero or more)
3. **Blank line:** `CRLF`
4. **Body:** Optional

Files to Create

src/response.rs

```
pub struct Response {
    pub status_code: u16,
    pub status_text: String,
    pub headers: Vec<(String, String)>,
    pub body: Option<Vec<u8>>,
}

impl Response {
    pub fn new(status_code: u16) -> Self {
        Self {
            status_code,
            status_text: status_text(status_code).to_string(),
            headers: Vec::new(),
            body: None,
        }
    }

    pub fn header(mut self, key: &str, value: &str) -> Self {
        self.headers.push((key.to_string(), value.to_string()));
        self
    }

    pub fn body(mut self, body: impl Into<Vec<u8>>) -> Self {
        let body_bytes = body.into();
        self.headers.push((
            "Content-Length".to_string(),
            body_bytes.len().to_string(),
        ));
        self.body = Some(body_bytes);
        self
    }

    pub fn to_bytes(&self) -> Vec<u8> {
        // Serialize the entire response into a byte vector
    }
}
```

Implementation Details

Status text lookup:

```
fn status_text(code: u16) -> &'static str {
    match code {
        200 => "OK",
        201 => "Created",
        204 => "No Content",
        301 => "Moved Permanently",
        302 => "Found",
        304 => "Not Modified",
        400 => "Bad Request",
        401 => "Unauthorized",
        403 => "Forbidden",
        404 => "Not Found",
        405 => "Method Not Allowed",
        413 => "Payload Too Large",
        500 => "Internal Server Error",
        501 => "Not Implemented",
        503 => "Service Unavailable",
        _ => "Unknown",
    }
}
```

Serialization (`to_bytes`):

```
pub fn to_bytes(&self) -> Vec<u8> {
    let mut response = format!(
        "HTTP/1.1 {} {}{}{}\\r\\n",
        self.status_code, self.status_text
    );

    for (key, value) in &self.headers {
        response.push_str(&format!("{}", key, value));
    }

    response.push_str("\\r\\n");

    let mut bytes = response.into_bytes();
    if let Some(ref body) = self.body {
        bytes.extend_from_slice(body);
    }
    bytes
}
```

Convenience constructors:

```
impl Response {
    pub fn ok() -> Self { Self::new(200) }
    pub fn not_found() -> Self { Self::new(404) }
    pub fn bad_request() -> Self { Self::new(400) }
    pub fn internal_error() -> Self { Self::new(500) }
    pub fn method_not_allowed() -> Self { Self::new(405) }
}
```

Sending the response over a `TcpStream`:

```

use std::io::Write;

impl Response {
    pub fn send(&self, stream: &mut impl Write) -> std::io::Result<()> {
        stream.write_all(&self.to_bytes())?;
        stream.flush()?;
        Ok(())
    }
}

```

Pitfalls

- **Always set `Content-Length`**. Without it, the client doesn't know when the response ends (unless using chunked transfer encoding, which is out of scope).
- **`Content-Type` matters**. If you send HTML without `Content-Type: text/html`, browsers will display it as plain text.
- **Don't forget the blank line**. The `\r\n\r\n` between headers and body is mandatory. Forgetting it is the #1 cause of "my server responds but the client hangs."
- **`flush()` is required**. `write_all` may buffer data internally. `flush()` ensures all bytes are actually sent over the wire.

Acceptance Criteria

- `Response::ok().body("Hello")` produces a valid HTTP response
- `Content-Length` is automatically set when a body is added
- `to_bytes()` produces correctly formatted HTTP/1.1 response bytes
- `send()` writes the response to a `TcpStream` successfully
- Unit tests verify serialization output for various status codes and bodies

SECTION 05

Phase 4: Routing

Timeline: 1-2 days **Difficulty:** 4/10

Objective

Build a router that maps incoming `(Method, Path)` pairs to handler functions. The router should support exact path matching and path parameters (e.g., `/users/:id`).

Key Concepts

- Function pointers and closures — handlers are functions passed as arguments
- `Fn` trait bounds — storing handler functions in a data structure
- Pattern matching on strings
- `HashMap` vs `Vec` for route storage

Files to Create

src/router.rs

```
use crate::request::{Request, Method};
use crate::response::Response;

pub type HandlerFn = fn(&Request) -> Response;

pub struct Route {
    pub method: Method,
    pub path_pattern: String,
    pub handler: HandlerFn,
}

pub struct Router {
    routes: Vec<Route>,
}

impl Router {
    pub fn new() -> Self {
        Self { routes: Vec::new() }
    }

    pub fn route(mut self, method: Method, path: &str, handler: HandlerFn) -> Self {
        self.routes.push(Route {
            method,
            path_pattern: path.to_string(),
            handler,
        });
        self
    }

    pub fn resolve(&self, request: &Request) -> Response {
        // 1. Find a route matching (method, path)
        // 2. If found, call the handler
        // 3. If path matches but method doesn't, return 405
        // 4. If nothing matches, return 404
    }
}
```

src/handlers.rs

```

use crate::request::Request;
use crate::response::Response;

pub fn index(_req: &Request) -> Response {
    Response::ok()
        .header("Content-Type", "text/html")
        .body("<h1>Welcome to Toy HTTP Server</h1>")
}

pub fn echo(req: &Request) -> Response {
    let body = req.body.clone().unwrap_or_default();
    Response::ok()
        .header("Content-Type", "text/plain")
        .body(body)
}

pub fn not_found(_req: &Request) -> Response {
    Response::not_found()
        .header("Content-Type", "text/plain")
        .body("404 - Not Found")
}

```

Implementation Details

Path matching with parameters:

A path pattern like `/users/:id` should match `/users/42` and extract `id = "42"`.

```

fn path_matches(pattern: &str, actual: &str) -> bool {
    let pattern_parts: Vec<&str> = pattern.split('/').collect();
    let actual_parts: Vec<&str> = actual.split('/').collect();

    if pattern_parts.len() != actual_parts.len() {
        return false;
    }

    pattern_parts.iter().zip(actual_parts.iter()).all(|(p, a)| {
        p.starts_with(':') || p == a
    })
}

```

Storing extracted path parameters on the Request:

Add a field to `Request`:

```

pub struct Request {
    // ... existing fields ...
    pub params: Vec<String, String>, // extracted path params
}

```

Extract them during routing:

```
fn extract_params(pattern: &str, actual: &str) -> Vec<(String, String)> {
    pattern.split('/')
        .zip(actual.split('/'))
        .filter_map(|(p, a)| {
            p.strip_prefix(':').map(|name| (name.to_string(), a.to_string()))
        })
        .collect()
}
```

The `resolve` method:

```
pub fn resolve(&self, request: &Request) -> Response {
    let mut path_matched = false;

    for route in &self.routes {
        if path_matches(&route.path_pattern, &request.path) {
            path_matched = true;
            if route.method == request.method {
                let mut req = request; // or clone
                return (route.handler)(&req);
            }
        }
    }

    if path_matched {
        Response::method_not_allowed()
            .header("Content-Type", "text/plain")
            .body("405 - Method Not Allowed")
    } else {
        Response::not_found()
            .header("Content-Type", "text/plain")
            .body("404 - Not Found")
    }
}
```

Pitfalls

- **Route order matters.** If you have `/users/:id` and `/users/admin`, put the more specific route first. Otherwise `:id` will match `admin`.
- **Trailing slashes.** Decide early: does `/users` match `/users/`? Pick one convention and be consistent.
- **Handler function signatures.** Using `fn(&Request) -> Response` (function pointer) is simpler than `Box<dyn Fn(&Request) -> Response>` (trait object). Start with function pointers; upgrade to trait objects later if you need closures.

Acceptance Criteria

- Router correctly dispatches GET `/` to the index handler
- Router correctly dispatches POST `/echo` to the echo handler
- Router returns 404 for unmatched paths
- Router returns 405 when path matches but method doesn't
- Path parameters are extracted and accessible (e.g., `/users/:id` with `/users/42` yields `id = "42"`)
- Unit tests cover: exact match, parameter match, 404, 405

SECTION 06

Phase 5: Multi-threading

Timeline: 1 day Difficulty: 4/10

Objective

Handle multiple client connections simultaneously by spawning a new OS thread for each connection. This is the simplest concurrency model and a stepping stone to the thread pool.

Key Concepts

- `std::thread::spawn` — creates a new OS thread
- `Send` trait — types that can be transferred across thread boundaries
- `move` closures — transferring ownership into a thread
- Why unbounded thread creation is dangerous (resource exhaustion)

Files to Modify

`src/server.rs`

```
pub fn run(&self) -> Result<(), std::io::Error> {
    for stream in self.listener.incoming() {
        match stream {
            Ok(mut stream) => {
                let router = self.router.clone(); // Router must impl Clone
                std::thread::spawn(move || {
                    handle_connection(&mut stream, &router);
                });
            }
            Err(e) => {
                eprintln!("Failed to accept connection: {}", e);
            }
        }
    }
    Ok(())
}
```

Connection handler function:

```
fn handle_connection(stream: &mut TcpStream, router: &Router) {
    let mut buffer = [0u8; 4096];
    match stream.read(&mut buffer) {
        Ok(bytes_read) => {
            if bytes_read == 0 { return; }
            match Request::parse(&buffer[..bytes_read]) {
                Ok(request) => {
                    let response = router.resolve(&request);
                    let _ = response.send(stream);
                }
                Err(_) => {
                    let response = Response::bad_request()
                        .header("Content-Type", "text/plain")
                        .body("400 - Bad Request");
                    let _ = response.send(stream);
                }
            }
        }
        Err(e) => {
            eprintln!("Failed to read from stream: {}", e);
        }
    }
}
```

Implementation Details

Why `move` is required:

`thread::spawn` takes a `FnOnce() + Send + 'static` closure. The `move` keyword transfers ownership of `stream` and `router` into the closure. Without it, the closure would borrow them, but the compiler can't guarantee the borrows outlive the spawning thread.

Why `Router` needs `Clone`:

Each thread needs its own copy of the router (or a shared reference via `Arc`). The simplest approach is to derive `Clone` on `Router` and `Route`. Alternatively, wrap the router in `Arc<Router>` and clone the `Arc` (cheaper — just increments a reference count).

`TcpStream` is `Send`:

`TcpStream` implements `Send`, which means it can be moved into a new thread. This is essential — without it, you couldn't handle the connection on a different thread.

Pitfalls

- **Unbounded threads = denial of service.** Each thread consumes ~2-8 MB of stack space. A malicious client opening 10,000 connections will exhaust memory. This is why Phase 6 (thread pool) exists.
- **No connection timeout.** A slow client can hold a thread indefinitely. For now, this is acceptable. A production server would set read/write timeouts.
- **Thread creation overhead.** Spawning a thread per connection is expensive (~microseconds). For a toy server, this is fine. For a production server, you'd use a thread pool or async I/O.

Acceptance Criteria

- Server handles multiple concurrent `curl` requests without blocking
- Opening 5 terminal tabs and running `curl` simultaneously works correctly
- No panics or crashes when a client disconnects mid-request
- Each connection is handled in its own thread (verify with print statements showing thread IDs)

SECTION 07

Phase 6: Thread Pool

Timeline: 3-4 days **Difficulty:** 7/10

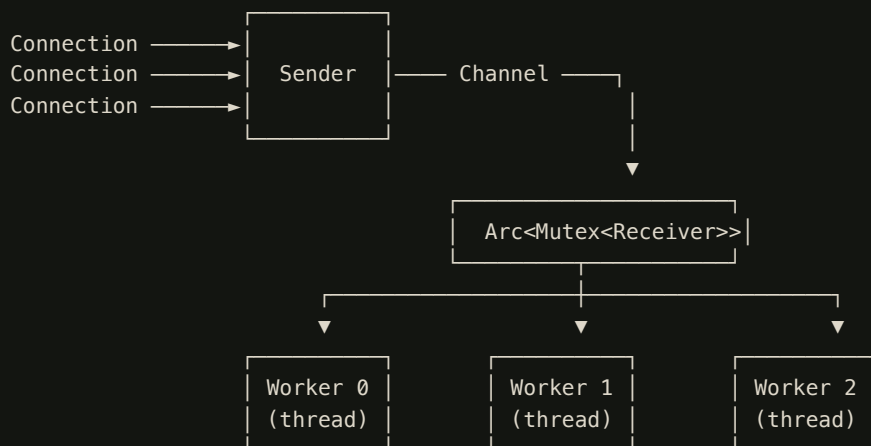
Objective

Replace unbounded thread spawning with a fixed-size pool of worker threads. Workers pull jobs from a shared queue using channels. This is the hardest phase and the biggest learning opportunity.

Key Concepts

- `std::sync::mpsc` — multi-producer, single-consumer channels
- `std::sync::Arc` — atomic reference counting for shared ownership across threads
- `std::sync::Mutex` — mutual exclusion for safe shared mutable state
- `Drop` trait — cleanup logic when a value goes out of scope (graceful shutdown)
- `Send + 'static` bounds on thread closures

Architecture



Files to Create

```
src/thread_pool.rs
```

```
use std::sync::{Arc, Mutex, mpsc};
use std::thread;

type Job = Box<dyn FnOnce() + Send + 'static>;

pub struct ThreadPool {
    workers: Vec<Worker>,
    sender: Option<mpsc::Sender<Job>>,
}

struct Worker {
    id: usize,
    thread: Option<thread::JoinHandle<()>>,
}

impl ThreadPool {
    pub fn new(size: usize) -> Self {
        assert!(size > 0);

        let (sender, receiver) = mpsc::channel();
        let receiver = Arc::new(Mutex::new(receiver));

        let mut workers = Vec::with_capacity(size);
        for id in 0..size {
            workers.push(Worker::new(id, Arc::clone(&receiver)));
        }

        Self {
            workers,
            sender: Some(sender),
        }
    }

    pub fn execute<F>(&self, f: F)
    where
        F: FnOnce() + Send + 'static,
    {
        let job = Box::new(f);
        self.sender.as_ref().unwrap().send(job).unwrap();
    }
}

impl Worker {
    fn new(id: usize, receiver: Arc<Mutex<mpsc::Receiver<Job>>>) -> Self {
        let thread = thread::spawn(move || loop {
            let message = receiver.lock().unwrap().recv();

            match message {
                Ok(job) => {
                    println!("Worker {id} executing a job");
                    job();
                }
                Err(_) => {
                    println!("Worker {id} disconnected, shutting down");
                    break;
                }
            }
        });

        Self {
            id,
            thread: Some(thread),
        }
    }
}
```

```

        }
    }
});

Self {
    id,
    thread: Some(thread),
}
}
}

```

Implementation Details

Why `Arc<Mutex<Receiver>>`?

`mpsc::Receiver` is not `Clone` — it's single-consumer by design. But we have multiple worker threads that all need to pull from the same receiver. `Arc` provides shared ownership, and `Mutex` ensures only one worker receives each job.

Why `Option<thread::JoinHandle<>>`?

We need to take ownership of the `JoinHandle` during shutdown (to call `.join()`). `Option` lets us use `.take()` to move the handle out of the `Worker` struct.

Why `Option<mpsc::Sender<Job>>`?

Same reason — we need to drop the sender during shutdown to signal workers to stop. Dropping the sender causes `recv()` to return `Err`, which breaks the worker loop.

Graceful shutdown via `Drop`:

```

impl Drop for ThreadPool {
    fn drop(&mut self) {
        // Drop the sender first to close the channel
        drop(self.sender.take());

        println!("Shutting down all workers");

        for worker in &mut self.workers {
            if let Some(thread) = worker.thread.take() {
                println!("Shutting down worker {}", worker.id);
                thread.join().unwrap();
            }
        }
    }
}

```

Updated `server.rs`:

```
pub fn run(&self) -> Result<(), std::io::Error> {
    let pool = ThreadPool::new(4); // 4 worker threads

    for stream in self.listener.incoming() {
        match stream {
            Ok(stream) => {
                let router = Arc::clone(&self.router);
                pool.execute(move || {
                    handle_connection(&mut stream.clone(), &router);
                    // or pass stream by value into the closure
                });
            }
            Err(e) => {
                eprintln!("Failed to accept connection: {}", e);
            }
        }
    }
    Ok(())
}
```

Pitfalls

- **Mutex poisoning.** If a worker thread panics while holding the lock, the `Mutex` becomes “poisoned” and subsequent `.lock()` calls return `Err`. For a toy server, `.unwrap()` is acceptable. Production code should handle poisoning.
- **Deadlocks.** Never hold the `Mutex` lock across a `.recv()` call. Lock, receive, then release. The code above does this correctly: `receiver.lock().unwrap().recv()` locks, calls `recv`, and the temporary `MutexGuard` is dropped immediately.
- **`FnOnce` vs `Fn` vs `FnMut`.** Jobs must be `FnOnce` because each job runs exactly once. `Box<dyn FnOnce(>` is the correct type.
- **Thread count.** A good default is the number of CPU cores (`std::thread::available_parallelism()`), but for a toy server, 4 is fine.

Acceptance Criteria

- Server starts with a fixed number of worker threads (e.g., 4)
- Incoming connections are distributed across workers via the channel
- Server handles concurrent requests correctly (no data races, no crashes)
- Sending more concurrent requests than workers results in queuing (not crashes)
- Graceful shutdown: when the server is dropped, all workers finish their current job and exit cleanly
- Unit tests verify: pool creation, job execution, shutdown behavior

SECTION 08

Phase 7: Polish and Hardening

Timeline: 2-3 days Difficulty: 5/10

Objective

Add proper error handling, logging, edge case handling, and integration tests. Make the server robust and production-adjacent.

7A: Custom Error Types

Create a unified error type for the server:

```

#[derive(Debug)]
pub enum ServerError {
    Io(std::io::Error),
    Parse(ParseError),
    Route(String),
}

impl std::fmt::Display for ServerError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            Self::Io(e) => write!(f, "IO error: {}", e),
            Self::Parse(e) => write!(f, "Parse error: {:?}", e),
            Self::Route(msg) => write!(f, "Route error: {}", msg),
        }
    }
}

impl std::error::Error for ServerError {}

impl From<std::io::Error> for ServerError {
    fn from(e: std::io::Error) -> Self { Self::Io(e) }
}

impl From<ParseError> for ServerError {
    fn from(e: ParseError) -> Self { Self::Parse(e) }
}

```

7B: Request Logging

Add simple request logging to `handle_connection`:

```
fn handle_connection(mut stream: TcpStream, router: &Router) {
    let mut buffer = [0u8; 4096];
    match stream.read(&mut buffer) {
        Ok(bytes_read) if bytes_read > 0 => {
            match Request::parse(&buffer[..bytes_read]) {
                Ok(request) => {
                    println!("{}",
                        chrono_or_timestamp(),
                        request.method,
                        request.path
                    );
                    let response = router.resolve(&request);
                    let _ = response.send(&mut stream);
                }
                Err(e) => {
                    eprintln!("[ERROR] Failed to parse request: {:?}", e);
                    let response = Response::bad_request()
                        .body("400 - Bad Request");
                    let _ = response.send(&mut stream);
                }
            }
        }
        Ok(_) => {} // 0 bytes = client disconnected
        Err(e) => eprintln!("[ERROR] Read failed: {}", e),
    }
}
```

For timestamps, use `std::time::SystemTime` to avoid external dependencies:

```
fn timestamp() -> String {
    use std::time::SystemTime;
    let duration = SystemTime::now()
        .duration_since(SystemTime::UNIX_EPOCH)
        .unwrap();
    format!("{}", duration.as_secs())
}
```

7C: Edge Cases to Handle

- Empty requests** — client connects and immediately disconnects (0 bytes read). Handle gracefully, no panic.
- Oversized requests** — request exceeds the 4096-byte buffer. Options:
 - Return `413 Payload Too Large`
 - Loop reads until `\r\n\r\n` is found, then read `Content-Length` bytes for the body
- Keep-alive connections** — HTTP/1.1 defaults to keep-alive. For simplicity, add `Connection: close` to every response:

```
response.header("Connection", "close")
```

- HEAD requests** — same as GET but no body in the response. Handle in the router or response layer.
- Unknown methods** — return `501 Not Implemented`.

7D: Integration Tests

tests/integration_test.rs

```
use std::io::{Read, Write};
use std::net::TcpStream;
use std::thread;
use std::time::Duration;

fn start_test_server() {
    thread::spawn(|| {
        // Start server on a test port (e.g., 8081)
    });
    thread::sleep(Duration::from_millis(100)); // Wait for server to start
}

#[test]
fn test_get_request() {
    start_test_server();

    let mut stream = TcpStream::connect("127.0.0.1:8081").unwrap();
    stream.write_all(b"GET / HTTP/1.1\r\nHost: localhost\r\n\r\n").unwrap();

    let mut response = String::new();
    stream.read_to_string(&mut response).unwrap();

    assert!(response.contains("HTTP/1.1 200 OK"));
}

#[test]
fn test_404_response() {
    // Send request to non-existent path
    // Assert response contains "404"
}

#[test]
fn test_post_echo() {
    // Send POST with body
    // Assert response body matches request body
}

#[test]
fn test_concurrent_requests() {
    // Spawn 10 threads, each sending a request
    // Assert all 10 receive valid responses
}
```

Acceptance Criteria

- Custom error types replace raw `unwrap()` calls in non-test code
- Every request is logged with method, path, and timestamp
- Empty requests and client disconnects don't crash the server
- All responses include `Connection: close` header
- Integration tests pass for GET, POST, 404, 405, and concurrent requests
- Server runs stably under `for i in $(seq 1 100); do curl localhost:8080 & done`

SECTION 09

Final Checklist

Before considering the project complete, verify:

- ☐ Server starts and listens on a configurable port
- ☐ Handles GET, POST, PUT, DELETE requests
- ☐ Parses headers, query strings, path parameters, and request bodies
- ☐ Routes requests to the correct handler based on method and path
- ☐ Returns proper HTTP status codes (200, 201, 400, 404, 405, 500)
- ☐ Handles multiple concurrent connections via thread pool
- ☐ Shuts down gracefully (no zombie threads)
- ☐ Has unit tests for request parsing, response building, and routing
- ☐ Has integration tests with real TCP connections
- ☐ No `unwrap()` in production code paths (only in tests)
- ☐ `cargo build` and `cargo test` both pass cleanly
- ☐ `cargo clippy` produces no warnings

SECTION 10

Suggested Extensions (Post-MVP)

These are optional enhancements if you want to keep going after the core build:

1. **Static file serving** — serve files from a `public/` directory (teaches filesystem I/O and MIME types)
2. **Query string parsing** — parse `?key=value&foo=bar` into a `HashMap`
3. **JSON support** — add `serde` and `serde_json` for JSON request/response bodies
4. **Middleware** — chain handlers (e.g., logging middleware, auth middleware)
5. **Async with Tokio** — rewrite the server using `tokio` for async I/O (massive performance improvement, different concurrency model)
6. **HTTPS** — add TLS support with `rustls` (teaches certificate handling and encryption basics)